Bloom & Grind — Specification (P2)

Nov 2, 2026 · @Proximiti.

1. Overview

Bloom & Grind is an online store for specialty coffee and brewing gear, aimed at home brewers aged 20–35. This document freezes what we will deliver at P3: requirements A1–A12 and bonuses B1, B2, B3 and B6 (18 points).

Team (group CEN4.H A). Avram Eliana owns A1, A2, A3, A8, A9, A11, B1 and B3. Ferraglio Danut owns A4, A5, A6, A7, A10, A12, B2 and B6.

Stack. Next.js with TypeScript, React and Tailwind CSS, using its own API route handlers; PostgreSQL through the Prisma ORM; Auth.js (credentials) with bcrypt; Stripe Checkout in test mode; Nodemailer with Mailtrap; Vitest; Docker Compose.

Data. A seed script loads 35 products in 5 categories (filter coffee, espresso coffee, brewers, grinders and scales, accessories), each with images, price and stock, plus 2 delivery methods and the test accounts. Prices are stored as whole bani (1 leu = 100 bani).

Repository. github.com/Proximito/Bloom-and-Grind, commit tagged P2. Secrets live in `.env`; the repository holds only `.env.example`.

2. Requirements A1–A12

Every mandatory requirement has one owner and one implementation, summarised below in 1–2 sentences.

- **A1 Product catalog** (Eliana). Pages /products (24 per page via Prisma `skip` / `take`), /category/[slug] and /product/[slug] with name, description, price, image gallery and availability (in stock, only N left, out of stock).
- **A2 Basic search** (Eliana). Case-insensitive match on product names (Prisma `contains`); no results shows “No products match …” with a link back to the catalog.
- **A3 Shopping cart** (Eliana). Cart and CartItem rows per logged-in customer, so the cart survives logout; guests go to login. Each change recalculates the total on the server from database prices and rejects quantities above stock.

- **A4 User accounts** (Danut). Register, log in and log out with Auth.js (credentials); passwords hashed with bcrypt, cost 12. Customers save addresses and mark one as default.
- **A5 Checkout** (Danut). Address → delivery method (home courier 20 lei or parcel locker 12 lei, free from 250 lei) → summary with server totals → Stripe Checkout. The confirmation page shows the order number (e.g. BG-1042).
- **A6 Sandbox payment** (Danut). Orders start NEW (unpaid). Only the `checkout.session.completed` webhook, verified by Stripe signature, sets PAID; the success page only reads status. A declined card leaves the order unpaid.
- **A7 Stock management** (Danut). In the webhook transaction each item runs `UPDATE … SET stock = stock − qty WHERE stock >= qty`; if one fails, everything rolls back and the order is cancelled and refunded. Cancelling a paid order restores stock.
- **A8 Order history** (Eliana). /account/orders lists number, date, total and status; a detail page lists products. Every query filters by the session user id.
- **A9 Administration panel** (Eliana). Our own /admin for role ADMIN: product and category CRUD with image upload, orders filtered by status. NEW → PAID comes only from the webhook; admins set PAID → SHIPPED → DELIVERED, or cancel NEW or PAID orders.
- **A10 Basic security** (Danut). Zod validates all input on the server; all queries go through Prisma. Middleware and handlers return 403 on /admin and /api/admin for non-admins; cross-origin writes are rejected (CSRF); cookies are httpOnly, SameSite=Lax.
- **A11 Responsive design** (Eliana). Mobile-first Tailwind, checked at 360 px: catalog grid 1 → 2 → 4 columns, cart and checkout in one column, filters behind a button.
- **A12 Automated tests** (Danut). Vitest, `npm test`, at least 12 tests: add to cart, quantity above stock, totals with delivery and free threshold, webhook paid, repeated webhook, declined payment, last-unit race, cancel restores stock, admin API 403.

3. Bonus features

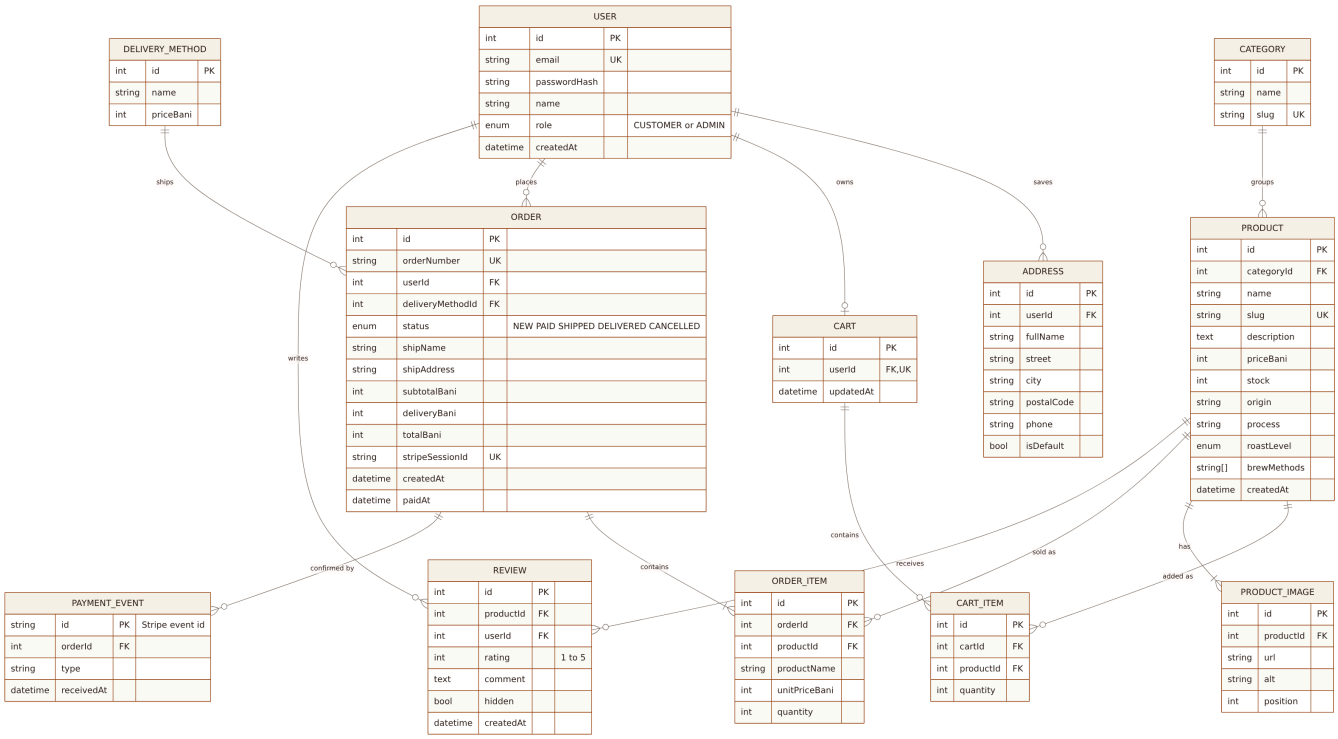
We ask for approval of four bonuses worth 18 points, built in week 11 after A1–A12 work.

- **B1 Filtered search, 5 pts** (Eliana). Combinable filters by category, price range, availability and brew method, sorting by price and date added; all kept in URL query parameters.

- **B2 Recommendations, 4 pts** (Danut). “Frequently bought together” from products sharing paid orders; with too few orders, “Similar products” by category and shared brew methods. Nothing is chosen by hand.
- **B3 Reviews and ratings, 4 pts** (Eliana). Only customers with a paid, shipped or delivered order containing the product can rate it 1–5, once. The average shows in the catalog; admins can hide a review.
- **B6 Transactional email, 5 pts** (Danut). When the webhook marks an order PAID, Nodemailer sends the confirmation (number, items, total, address) to the Mailtrap sandbox.

4. Data model

Twelve PostgreSQL tables cover accounts, catalog, cart, orders, payments and reviews; Prisma migrations create them and the seed fills them.

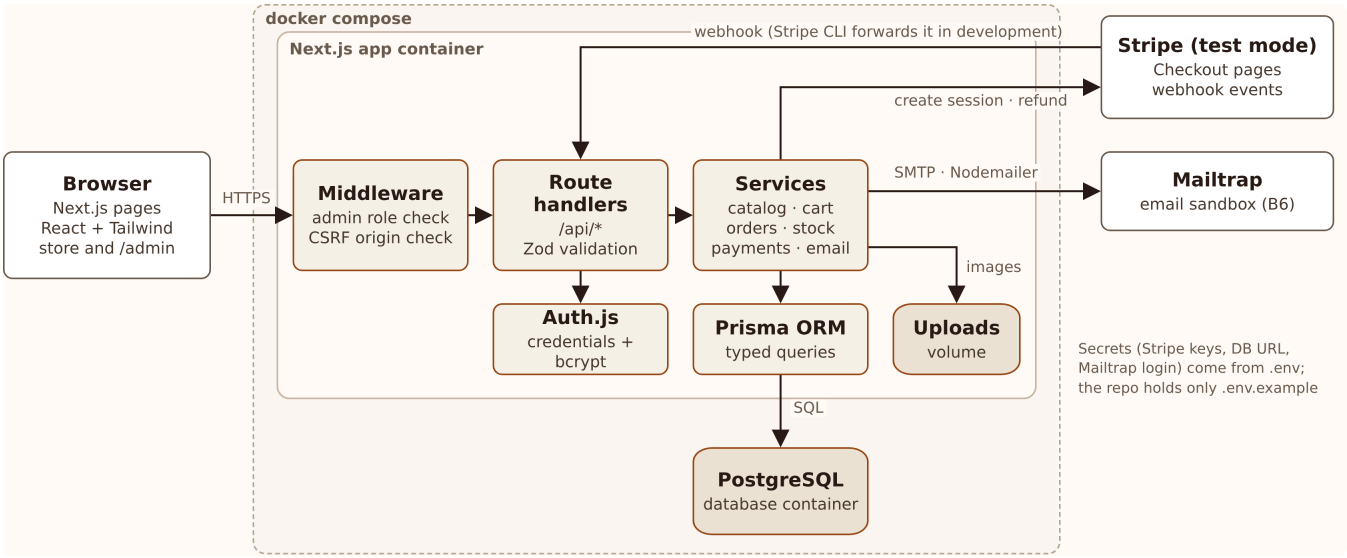


- **User, Address** (A4): `email` unique; `role` CUSTOMER or ADMIN; one default address per user.
- **Cart, CartItem** (A3): one cart per user; one row per product in a cart (`cartId`, `productId` unique).
- **Category, Product, ProductImage** (A1, A2, A9, B1): unique slugs; `priceBani` and `stock` are integers ⩾ 0; `brewMethods` feeds filters and B2.

- **DeliveryMethod** (A5): home courier 2000 bani, parcel locker 1200 bani.
- **Order, OrderItem** (A5–A9): unique `orderNumber` and `stripeSessionId`; `status` NEW, PAID, SHIPPED, DELIVERED or CANCELLED; OrderItem copies name and unit price at purchase.
- **PaymentEvent** (A6, A7): primary key = Stripe event id, so a repeated webhook is ignored.
- **Review** (B3): `rating` 1–5; one per (`productId, userId`); `hidden` set by admins.

5. Architecture

One Next.js application serves both the pages and the API; it talks to PostgreSQL through Prisma and to two outside services, Stripe and Mailtrap.



Every request passes middleware (admin role, CSRF origin check), then a route handler (Zod validation), then a service (business rules), then Prisma. In development the Stripe CLI (`stripe listen`) forwards webhooks to the local app.

API endpoints

Public = no login; Customer = any logged-in user; Owner = the record must belong to the session user (otherwise 404); Admin = role ADMIN (otherwise 403). CRUD = GET, POST, PUT/PATCH and DELETE on the collection and its [id].

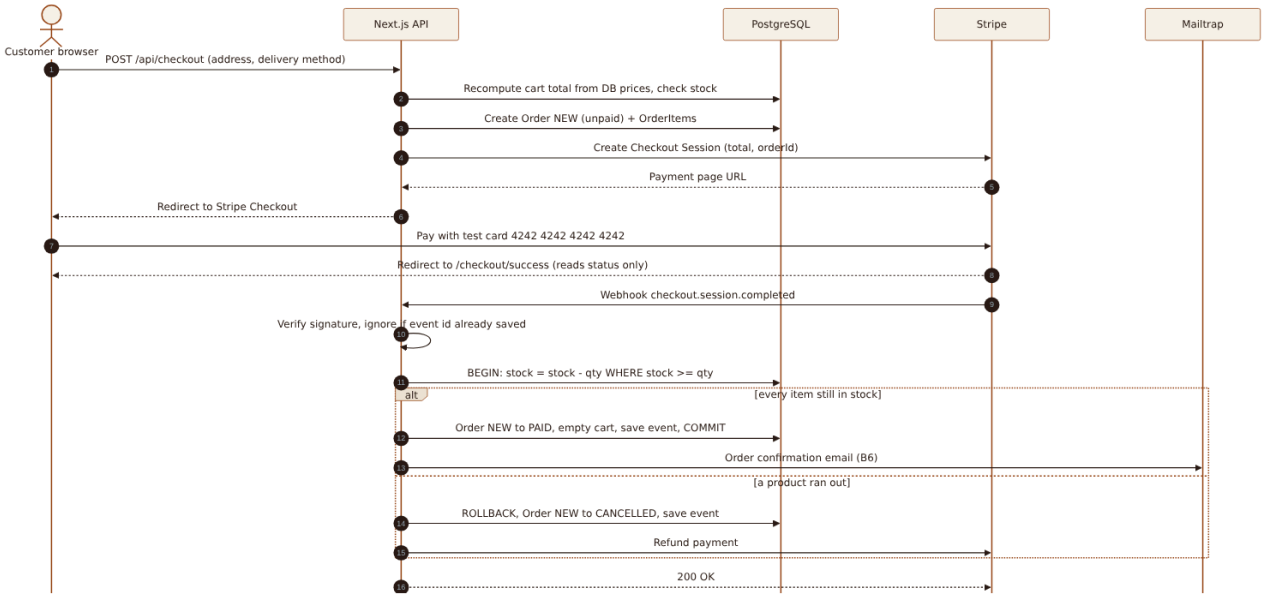
Method	Route	Access	Covers
GET	/api/products (page, q, category, price, inStock, brew, sort)	Public	A1, A2, B1
GET	/api/products/[slug], /api/categories	Public	A1
GET	/api/products/[id]/recommendations	Public	B2
GET, POST	/api/products/[id]/reviews	Public, buyer	B3
POST	/api/auth/register, /api/auth/[...nextauth]	Public	A4
CRUD	/api/account/addresses[/id]	Owner	A4
CRUD	/api/cart, /api/cart/items[/productId]	Customer	A3
GET	/api/delivery-methods	Public	A5
POST	/api/checkout	Customer	A5, A6
GET	/api/orders, /api/orders/[id]	Owner, Admin	A8
POST	/api/webhooks/stripe	Stripe signature	A6, A7, B6
CRUD	/api/admin/products[/id], /api/admin/categories[/id]	Admin	A9
POST	/api/admin/uploads	Admin	A9
GET	/api/admin/orders?status=	Admin	A9
PATCH	/api/admin/orders/[id]/status	Admin	A7, A9
PATCH	/api/admin/reviews/[id]	Admin	B3

6. Critical flow: payment

An order becomes PAID only when Stripe’s signed webhook arrives, and its stock drops in the same database transaction.

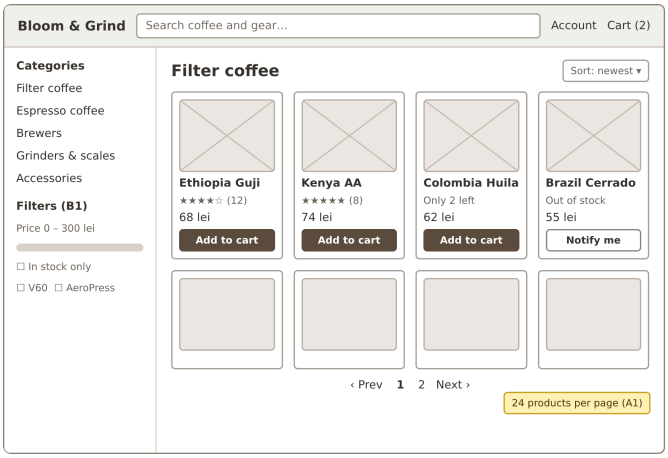
- **Declined card** (4000 0000 0000 0002): no completed event arrives, so the order stays NEW (unpaid) and stock is unchanged.

- **Tab closed after paying:** the webhook still arrives and marks the order PAID.
- **Same event twice:** its id is already in PaymentEvent, so the handler returns 200 and changes nothing.
- **Two buyers, last unit:** only one conditional update finds enough stock; the other order is rolled back, cancelled and refunded.
- **Forged webhook:** the signature check fails, the handler returns 400 and changes nothing.
- **Admin cancels a PAID order:** one transaction sets CANCELLED and restores stock; Stripe refunds the payment.



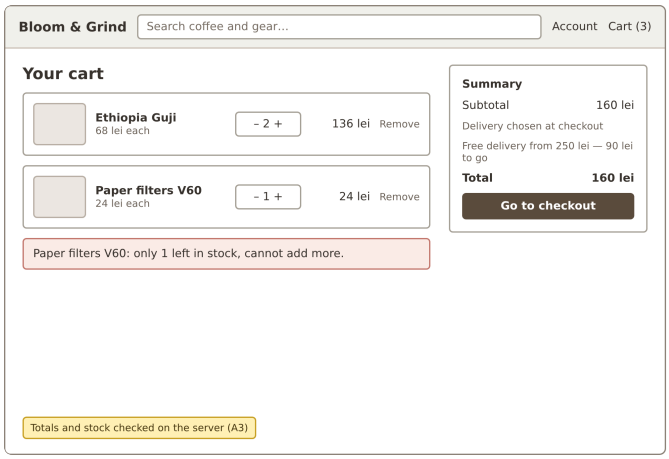
7. User interface

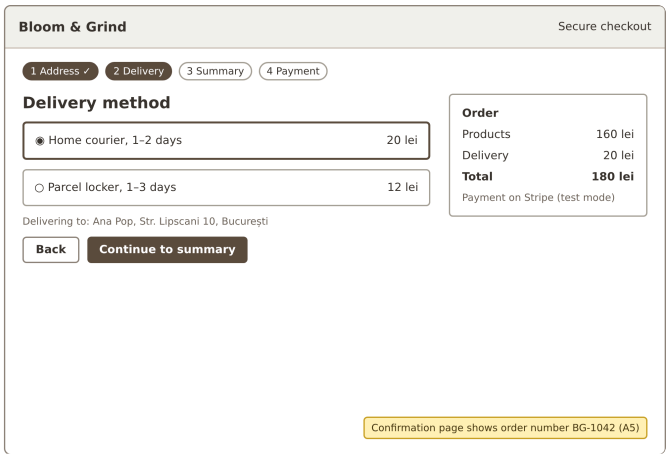
Low-fidelity wireframes fix the layout and content of the five required screens; colours and type come later.



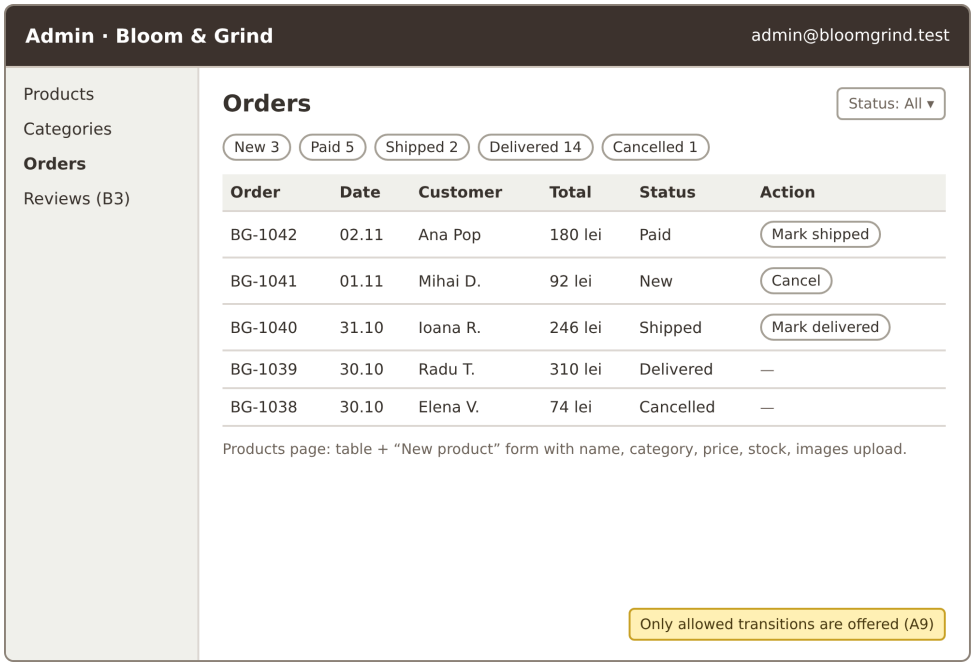


Catalog (A1, A2, B1, B3): categories and filters on the left, 24 products per page, rating on each card. **Product page** (A1, B2): gallery, price, roast and brew-method tags, availability, quantity, “Frequently bought together”.





Cart (A3): quantity controls, line totals, the server’s stock message, free-delivery progress. **Checkout** (A5): four steps; step 2 offers home courier (20 lei) or parcel locker (12 lei) with the total updated.





Administration panel (A9, B3): Products, Categories, Orders and Reviews; the order list filters by status and offers only the allowed next step. **Phone, 360 px** (A11): one column, filters and sort behind buttons.

8. Working prototype

At P2 we show the catalog read live from PostgreSQL and the login, running from the commit tagged P2.

1. `docker compose up` starts the app and PostgreSQL; migrations and the seed load 35 products in 5 categories.
2. /products shows page 1 and page 2, a category page, a product page, and a search that finds nothing.
3. A new customer registers, logs out and logs back in; the password is stored as a bcrypt hash.
4. Logged in as the customer, /admin returns 403; logged in as the admin, the panel opens.

Running it (5 commands):

```
git clone https://github.com/Proximito/Bloom-and-Grind.git
cd Bloom-and-Grind
cp .env.example .env
docker compose up --build
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

The last command is needed only for payments (from week 9). Test accounts are seeded: customer@bloomgrind.test and admin@bloomgrind.test; their passwords are in the README.